Agenda

- Functional Programming
- Destructuring
- Spread and Rest Operator
- DOM
- ~~AJAX~~
- ~~Promise~~
- ~~Node~~

Functional Programming

- map(fn) : operates on each array element and collect results into a new array.

```
function sqrNumsMap() {
let nums = [11, 22, 33, 44, 55];
let sqrs = nums.map((n) => n * n);
console.log("nums : " + nums);
console.log("sqrs : " + sqrs);
}
sqrNumsMap();
```

- filter(fn) : test a condition on each array element and collect elems satisfying the condition into a new array.

```
function numsEvenFilter() {
let nums = [11, 22, 33, 44, 55, 66, 77, 88];
let evens = nums.filter((n) => n % 2 == 0);
console.log("nums : " + nums);
console.log("evens : " + evens);
}
numsEvenFilter();
```

- flatMap(fn) : operates on each array element so that each element produce a new array and then collect results into a new array (flattened).

```
const words = ['hello world', 'how are you'];

// const individualWords = words.map(w => w.split(' '));
// console.log(individualWords)
// o/p -> [ [ 'hello', 'world' ], [ 'how', 'are', 'you' ] ]

const individualWords = words.flatMap(w => w.split(' '));
console.log(individualWords)
// o/p -> [ 'hello', 'world', 'how', 'are', 'you' ]
```

SUNBEAM INFOTECH

- reduce(fn, start) - calls given fn repeatedly with accumulated result and next element to produce a single final result.

```
function reduceSum() {
    let nums = [11, 22, 33, 44, 55];
    let total = nums.reduce((x, y) => x + y, 0);
    console.log("nums : " + nums);
    console.log("total : " + total);
}
reduceSum();
```

- sort(fn) - sorts given array with order defined by given fn and return result as new array.

```
function sortNums() {
    let nums = [44, 33, 66, 22, 11, 55];
    let sorted = nums.sort();
    console.log("nums : " + nums);
    console.log("sorted : " + sorted);
    let revsorted = nums.sort((x, y) => y - x);
    console.log("rev sorted : " + revsorted);
}
sortNums();
```

String

- Strings can be enclosed in double quotes, single quotes, or back quotes (template strings).
- ${expr} in template strings is evaluated dynamically and added in string.

```
const s1 = "Sunbeam";
const s2 = 'DMC';
const s3 = `${s1}, ${s2}!!`;
console.log(s1);
console.log(s2);
console.log(s3);
```

- length property returns number of chars in string.

```
const text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
console.length(text.length); // 26
```

- Escape characters like \n, \r, \t, \b, ', ", \ are supported (like Java).
- String objects can be created by a few ways:

SUNBEAM INFOTECH

```
const s1 = "Rohan";
console.log("s1 = " + s1 + " of type " + typeof s1); // string
const s2 = String(123);
console.log("s2 = " + s2 + " of type " + typeof s2); // string
const s3 = new String("Sunbeam");
console.log("s3 = " + s3 + " of type " + typeof s3); // object
```

- Not recommended to create String objects explicitly (with new).
  - The new keyword slows down execution speed.
  - String objects can produce unexpected results (equality check, eval(), etc).
- Strings can be compared using == (content check) or === (content and type check).
- Strings can be compared using == (address check) and returns false for two different objects (irrespective of contents).

```
const s4 = "Rohan";
console.log("s1 == s4 : ", s1 == s4); // true - two strings are equal
const s5 = new String("Sunbeam");
console.log("s3 == s5 : ", s3 == s5); // false - two objects are never equal (addr
comparison)
```

- Strings are immutable. Methods modifying string, returns new String objects.
- toUpperCase() and toLowerCase()

```
let text1 = "SunBeam";
let text2 = text1.toUpperCase();
let text3 = text1.toLowerCase();
```

- subscript vs at(index) vs charAt(index)
  - charAt() works similar to subscript operator.
  - at() added in ES 2022 and supports negative indexes too (like array)

```
const str = "sunbeam";
console.log("str[2] = " + str[2]); // n
console.log("str[-2] = " + str[-2]); // undefined
console.log("str.charAt(1) = " + str.charAt(1)); // u
console.log("str.charAt(-1) = " + str.charAt(-1)); // (blank "")
console.log("str.at(1) = " + str.at(1)); // u
console.log("str.at(-1) = " + str.at(-1)); // m
```

- slice(startIndex, endIndex) vs substring(startIndex, endIndex)
  - slice() extracts a part of a string and returns the extracted part in a new string.
  - startIndex is included and endIndex is excluded.
  - if endIndex is not given, returns the whole string from startIndex.

SUNBEAM INFOTECH

- startIndex and endIndex can be negative i.e. w.r.t. end of string.

```
let text = "0123456789";
let part = text.slice(2, 8);  // 234567

let text = "0123456789";
let part = text.slice(2);  // 23456789

let text = "0123456789";
let part = text.slice(-2);  // 89

let text = "0123456789";
let part = text.slice(-8, -2);  // 234567
```

- substring() is similar to slice().
  - if start and end index values less than 0, they are considered 0 in substring().

```
let text = "0123456789";
let part = text.substring(-2);  // 0123456789
```

- split() splits string into multiple parts.

```
let text = "DAC,DMC,DESD,DBDA,DITISS";
let parts = text.split(","); // ["DAC","DMC","DESD","DBDA","DITISS"]
```

Destructuring

- Destructuring is a shortcut syntax to unpack values from:
  1. Arrays → Extract elements by position
  2. Objects → Extract properties by name

1. Array Destructuring

```
const fruits = ["🍎", "🍌", "🍊"];
// Old way
const apple = fruits[0];
const banana = fruits[1];

// Destructuring
const [apple, banana, orange] = fruits;
console.log(apple); // "🍎"

// Skipping Elements
const [first, , third] = ["A", "B", "C"];
console.log(third); // "C"
```

SUNBEAM INFOTECH

```
//Default Values
const [a = 1, b = 2] = [10];
console.log(a, b); // 10, 2 (fallback for missing values)
```

2. Object Destructuring

```
const user = { name: "Rohan", age: 25 };

// Old way
const name = user.name;
const age = user.age;

// Destructuring
const { name, age } = user;
console.log(name); // "Rohan"

// Renaming Variables
const { name: userName, age: userAge } = user;
console.log(userName); // "Rohan"

// Default Values
const { role = "Guest" } = user;
console.log(role); // "Guest" (if missing in object)
```

The Spread Operator (...)

- The spread operator unpacks elements from arrays, objects, or other iterables.
- It is useful for creating new copies of data structures without changing the original.

- With Arrays

```
const fruits = ['apple', 'banana', 'cherry'];
const vegetables = ['carrot', 'broccoli'];

// Combine arrays
const produce = [...fruits, ...vegetables];
console.log(produce); // Output: ["apple", "banana", "cherry", "carrot",
"broccoli"]

// create a new array with a copy of an existing one.
const fruitsCopy = [...fruits];
fruitsCopy.push('grape');
console.log(fruits); // Output: ["apple", "banana", "cherry"]
console.log(fruitsCopy); // Output: ["apple", "banana", "cherry", "grape"]
```

- with Object

SUNBEAM INFOTECH

- The spread operator copies properties from one object to another.
- When merging objects, properties from the object that is spread last will overwrite earlier properties if they have the same key.

```
const person = { name: 'Alice', age: 30 };
const location = { city: 'New York', country: 'USA' };

// Merge objects
const profile = { ...person, ...location };
console.log(profile); // Output: { name: "Alice", age: 30, city: "New York",
country: "USA" }

// Update or override properties
const updatedProfile = { ...profile, age: 31 };
console.log(updatedProfile); // Output: { name: "Alice", age: 31, city: "New
York", country: "USA" }
```

- In function calls

```
 function add(n1, n2) {
    console.log(n1 + n2)
}

const arr = [10, 20]
add(...arr)
```

The Rest Operator (...)

The rest operator is used in function definitions to collect an indefinite number of arguments into a single array.

- It can also be used in destructuring assignments.

- In function parameters

- The rest parameter allows you to create functions that accept any number of arguments.
- The rest parameter must be the last parameter in the function definition.

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(1, 2, 3, 4, 5)); // Output: 15
console.log(sum(10, 20)); // Output: 30
```

- In array destructuring

SUNBEAM INFOTECH

- When destructuring an array, the rest operator can collect all remaining elements into a new array.

```
const [first, second, ...restOfTheNumbers] = [10, 20, 30, 40, 50];

console.log(first); // Output: 10
console.log(second); // Output: 20
console.log(restOfTheNumbers); // Output: [30, 40, 50]
```

- In object destructuring

- In object destructuring, the rest operator gathers the remaining properties into a new object.

```
const person = {
  name: 'Bob',
  age: 42,
  city: 'Chicago',
  job: 'Programmer'
};

const { name, ...otherInfo } = person;

console.log(name); // Output: "Bob"
console.log(otherInfo); // Output: { age: 42, city: "Chicago", job: "Programmer" }
```

SUNBEAM INFOTECH

Window Object

- It represents an open window in the browser. It is browser's object(not JS object) which is created automatically
- It is a global object with lot of properties and methods

DOM (Document Object Model)

- When a webpage is loaded the browser creates DOM of the page
- It is the data representation of the objects that comprise the structure and content of a document on the web.
- DOM represents an HTML document in memory
- The DOM represents the document as nodes and objects so that programming languages can interact with the page.
- In both cases, it is the same document but the Document Object Model (DOM) representation allows it to be manipulated.
- console.dir(document) will display all the properties and methods from the document
- It is a tree like structure (window-> document -> html -> and all its sub nodes)
- It Enables dynamic interaction with:
  - HTML elements
  - CSS styles
  - Event handling

- Content updates

DOM Traversal

Basic Navigation

```
// Parent/child relationships
document.childNodes; // NodeList of all children
document.head; // <head>
document.body; // <body>
document.body.parentNode; // <html>
document.body.firstChild; // First child node
document.body.lastChild; // Last child node
```

Modern Methods

```
// Get children (element nodes only)
document.children; // [<html>]
document.children[0].children; // [head,body]
document.firstElementChild.children; // [head,body]
document.firstElementChild.lastElementChild; // body
```

Selection

1. Selecting with id
   - document.getElementById("myId") (#)
2. Selecting with class
   - document.getElementsByClassName("myClass") (.)
   - returns HTML collection an array of objects
3. Selecting with tag
   - document.getElementsByTagName("tagName")
   - returns HTML collection an array of objects
4. Query Selector
   - used to select the id,name and class automatically
   - document.querySelector("myId/myClass/tag")
     - returns first element
   - document.querySelectorAll("myId/myClass/tag")
     - returns a NodeList

```
// Querying
document.getElementById("header");
document.getElementsByName("header");
document.getElementsByTagName("div");
document.getElementsByClassName("container");
```

SUNBEAM INFOTECH

```
document.querySelector(".main-content"); // Returns first match
document.querySelectorAll("button"); // Returns NodeList
```

DOM Manipulation

Creating Elements

```
// Create new element
const newDiv = document.createElement("div");
newDiv.textContent = "Hello World";
// Add to DOM
document.body.appendChild(newDiv);
// Insert at specific position
parent.insertBefore(newDiv, referenceElement);
```

Modifying Elements

1. Modifying contents

```
 // Content
element.innerHTML = "<strong>Warning!</strong>"; // Parses HTML
element.innerText = "Plain text only"; // Safer alternative
```

2. Modifying attributes

```
 // Attributes
element.attributeName = attrValue;
element.setAttribute("data-id", "123");
const id = element.getAttribute("data-id");
element.attributeName;
element.removeAttribute("data-id");
```

3. Modifying Styles

```
// Styles
element.style.color = "red";
element.style.fontSize = "16px"; // Use camelCase
```

4. Modifying style class

SUNBEAM INFOTECH

```
 // Classes
element.className = "myclass";
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("visible");
```

SUNBEAM INFOTECH